

DataSoluTech

Solutions de gestion de données et d'analyse



Migration | Scalabilité | Performance

Migration des Données Médicales vers MongoDB

Démarche Technique Complète et Déploiement

Sommaire

1. Contexte et Objectifs
2. Démarche Technique
3. Justification des Choix Techniques
4. Fonctionnalités
5. Développement & Tests unitaires
6. Tests d'intégration locaux
7. Validation Docker (tests)
8. Déploiement production
9. Pipeline CI/CD
10. Résultats Obtenus
11. Déploiement cloud

Contexte et Objectifs

Contexte



Migration des données médicales pour un client confronté à des problèmes de scalabilité.



Nécessité de garantir la sécurité, la performance et l'intégrité des données.

Objectifs



Automatiser la migration d'un dataset de données médicales (format CSV) vers une base MongoDB.
Valider la migration avec des tests rigoureux.



Tester et déployer la solution en local, puis sous Docker.



Explorer les options de déploiement sur AWS.

Objectifs



Automatiser la migration d'un dataset de données médicales (format CSV) vers une base MongoDB.

Valider la migration avec des tests rigoureux.

- Un nettoyage et une normalisation des données (noms, dates)
- Une optimisation via des index adaptés aux requêtes métier
- Un contrôle d'accès par rôles (RBAC) pour sécuriser l'accès aux données sensibles
- Une suite de tests unitaires et d'intégration automatisés

Enjeux



Scalabilité



Performance



Sécurité des données

Démarche Technique

Base de données

healthcare_dataset.csv

```
data > healthcare_dataset.csv > data
```

```
1 Name, Age, Gender, Blood Type, Medical Condition, Date of Admission, Doctor, Hospital, Insurance Provider, Billing Amount, Room Number, Admission Type, Discharge Date, Medication, Test Result
```

Base de données

healthcare_dataset.csv

```
data > healthcare_dataset.csv > data
1 Name, Age, Gender, Blood Type, Medical Condition, Date of Admission, Doctor, Hospital, Insurance Provider, Billing Amount, Room Number, Admission Type, Discharge Date, Medication, Test Result
```

Chaque document de la collection **dataset_donnees_medicales** représente un séjour hospitalier.

```
_id: ObjectId('69962d5f6f9c43eac2aa0540')
Name : " "
Age : 
Gender : "Male"
Blood Type : "O"
Medical Condition : " "
Date of Admission : 2024-01-01T00:00:00.000+00:00
Doctor : "M. S."
Hospital : "St. Mary's"
Insurance Provider : "Blue Cross"
Billing Amount : 1500
Room Number : 301
Admission Type : "Emergency"
Discharge Date : 2024-01-05T00:00:00.000+00:00
Medication : "Painkillers"
Test Results : "Normal"
```

Base de données

Chaque document de la collection `dataset_donnees_medicales` représente un séjour hospitalier

healthcare_dataset.csv

document

```
{
  "_id": "50a5c381-58862a2c9f76c34ac2a850467",
  "Name": " ",
  "Age": 1,
  "Gender": "Male",
  "Blood Type": " ",
  "Medical Condition": " ",
  "Date of Admission": "2014-01-01T00:00:00.000+00:00",
  "Doctor": " ",
  "Hospital": " ",
  "Insurance Provider": " ",
  "Billing Amount": 0,
  "Room Number": 0,
  "Admission Type": " ",
  "Discharge Date": "2014-01-01T00:00:00.000+00:00",
  "Medication": " ",
  "Test Results": " "
}
```

Base de données

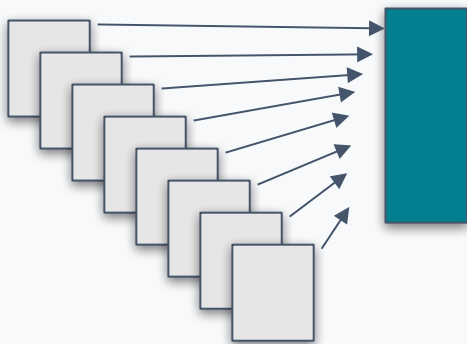
Chaque document de la collection `dataset_donnees_medicales` représente un séjour hospitalier

`healthcare_dataset.csv`

document

```
{
  "_id": "50a5c381-5882a2c6f7b0c3a02a824041",
  "Age": 65,
  "Gender": "Male",
  "Medical Condition": "Hypertension",
  "Date of Admission": "2014-01-01T00:00:00.000+00:00",
  "Discharge Date": "2014-01-03T00:00:00.000+00:00",
  "Discharge Type": "Discharge",
  "Billing Amount": 1500,
  "Insurance Provider": "Aetna",
  "Room Number": 101,
  "Nurse": "John Doe",
  "Medication": "Lisinopril",
  "Vital Signs": {
    "Heart Rate": 72,
    "Blood Pressure": 120/80,
    "Temperature": 98.6,
    "Respiratory Rate": 16,
    "Oxygen Saturation": 98
  }
}
```

collection



Base de données

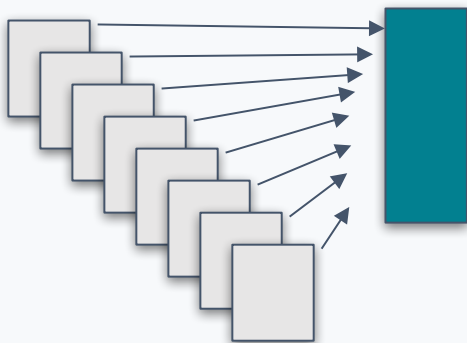
Chaque document de la collection `dataset_donnees_medicales` représente un séjour hospitalier

`healthcare_dataset.csv`

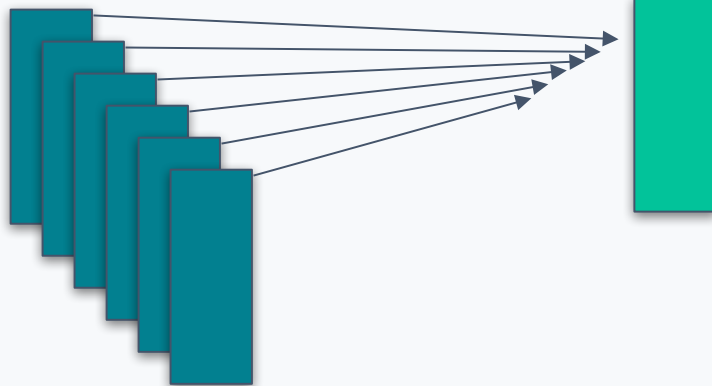
document

```
{
  "_id": "50a5c581-58862a5c767c3d3c2a85401",
  "Name": "John Doe",
  "Age": 45,
  "Gender": "Male",
  "Blood Type": "A",
  "Medical Condition": "Hypertension",
  "Date of Admission": "2024-01-01T00:00:00+00:00",
  "Discharge Date": "2024-01-05T00:00:00+00:00",
  "Doctor": "Dr. Smith",
  "Hospital": "St. Mary's Hospital",
  "Insurance Provider": "ABC Insurance",
  "Billing Amount": 1500,
  "Room Number": 101,
  "Admission Type": "Emergency",
  "Discharge Type": "Planned",
  "Medication": "Aspirin",
  "Vital Signs": {
    "Heart Rate": 75,
    "Blood Pressure": 120/80,
    "Temperature": 37.5,
    "Respiratory Rate": 16,
    "Oxygen Saturation": 98
  }
}
```

collection



database



Fonctionnalités et environnements

Fonctionnalités et environnements

transfert

optimisation

test intégrité

gestion des
utilisateurs

Fonctionnalités et environnements

transfert



local
=> dév. + tests

optimisation



Docker => test

test intégrité



Docker

gestion des
utilisateurs

Fonctionnalités et environnements

transfert

optimisation

test intégrité

gestion des
utilisateurs



local
=> dév. + tests



Docker => test



Docker

*Utilisation de variables
d'environnements*

.env.local

.env.test_docker

.env.docker

Étapes de Réalisation

Étapes de Réalisation

Étape 1 : Développement & Tests unitaires

Environnement : Local Python + venv

MongoDB : Non requis (mocks)

Étapes de Réalisation

Étape 1 : Développement & Tests unitaires

Environnement : Local Python + venv

MongoDB : Non requis (mocks)

Étape 2 : Tests d'intégration locaux

Environnement : Local Python + MongoDB local

MongoDB : Requis (localhost:27017)

Étapes de Réalisation

Étape 1 : Développement & Tests unitaires

Environnement : Local Python + venv

MongoDB : Non requis (mocks)

Étape 2 : Tests d'intégration locaux

Environnement : Local Python + MongoDB local

MongoDB : Requis (localhost:27017)

Étape 3 : Validation Docker (tests)

Environnement : Docker test (port 27018)

MongoDB : Docker mongodb_test

Étapes de Réalisation

Étape 1 : Développement & Tests unitaires

Environnement : Local Python + venv

MongoDB : Non requis (mocks)

Étape 2 : Tests d'intégration locaux

Environnement : Local Python + MongoDB local

MongoDB : Requis (localhost:27017)

Étape 3 : Validation Docker (tests)

Environnement : Docker test (port 27018)

MongoDB : Docker mongodb_test

Étape 4 : Déploiement production

Environnement : Docker production (port 27017)

MongoDB : Docker mongodb

Technologies utilisées

Catégorie	Technologie	Version	Rôle
Base de données	MongoDB	latest	Stockage NoSQL des dossiers médicaux
Langage	Python	3.11	Scripts de migration et tests
Conteneurisation	Docker	latest	Isolation et portabilité
Orchestration	Docker Compose	latest	Gestion multi-conteneurs
CI/CD	GitHub Actions	-	Tests automatisés sur push/PR
Tests	pytest	7.4.0+	Framework de tests
Sécurité	bcrypt	4.0.0+	Hashing mots de passe (12 rounds)
Data	pandas	2.3.3+	Manipulation des données CSV
BDD	pymongo	4.6.0+	Client MongoDB Python

Architecture

```
OC_P5/
├── .github/
│   └── workflows/
│       └── run_tests.yml           # Pipeline CI/CD GitHub Actions
├── data/
│   └── healthcare_dataset.csv      # Dataset source (55 500 dossiers)
├── scripts/                       # Scripts exécutables
│   ├── transfert_csv_mongodb.py   # Migration CSV → MongoDB
│   ├── optimiser_collection.py    # Création des 5 index + normalisation
│   ├── test_integrite_donnees.py  # Vérification intégrité CSV ↔ MongoDB
│   └── gestion_utilisateurs.py    # RBAC applicatif (bcrypt)
├── tests/                         # Tests pytest
│   ├── test_transfert_csv_mongodb.py # Tests unitaires migration
│   ├── test_optimiser_collection.py  # Tests unitaires optimisation
│   ├── test_test_integrite_donnees.py # Tests unitaires intégrité
│   ├── test_gestion_utilisateurs.py  # Tests unitaires RBAC applicatif
│   └── test_rbac_integration.py      # Tests intégration RBAC MongoDB
├── init-mongo.js                 # Initialisation RBAC MongoDB
├── mongod.conf                   # Configuration MongoDB (auth, profiling)
├── docker-compose.yml            # Production
├── docker-compose.test.yml       # Environnement de test
├── .env.example                   # Template variables d'environnement
├── requirements.txt               # Dépendances Python
└── README.md
```

Justification des Choix Techniques

Pourquoi MongoDB ?

Base de données

- Bases de données orientées documents
- 55,500 entrées.

Solutions apportées par MongoDB

Pourquoi MongoDB ?

Base de données

- Bases de données orientées documents
- 55,500 entrées.

Solutions apportées par MongoDB

- Flexibilité : structure non structurée ou semi-structurée, variabilité des données

Flexibilité

Pourquoi MongoDB ?

Base de données

- Bases de données orientées documents
- 55,500 entrées.

Solutions apportées par MongoDB

- Flexibilité : structure non structurée ou semi-structurée, variabilité des données
- Scalabilité horizontale : volume de données
=> conçu pour gérer de grands volumes de données et peut facilement être mis à l'échelle horizontalement en ajoutant plus de serveurs à un cluster.

Flexibilité

Scalabilité

Pourquoi MongoDB ?

Base de données

- Bases de données orientées documents
- 55,500 entrées.

Solutions apportées par MongoDB

- Flexibilité : structure non structurée ou semi-structurée, variabilité des données
- Scalabilité horizontale : volume de données => conçu pour gérer de grands volumes de données et peut facilement être mis à l'échelle horizontalement en ajoutant plus de serveurs à un cluster.
- Performance : index appropriés
- Répartition de la Charge

Flexibilité

Scalabilité

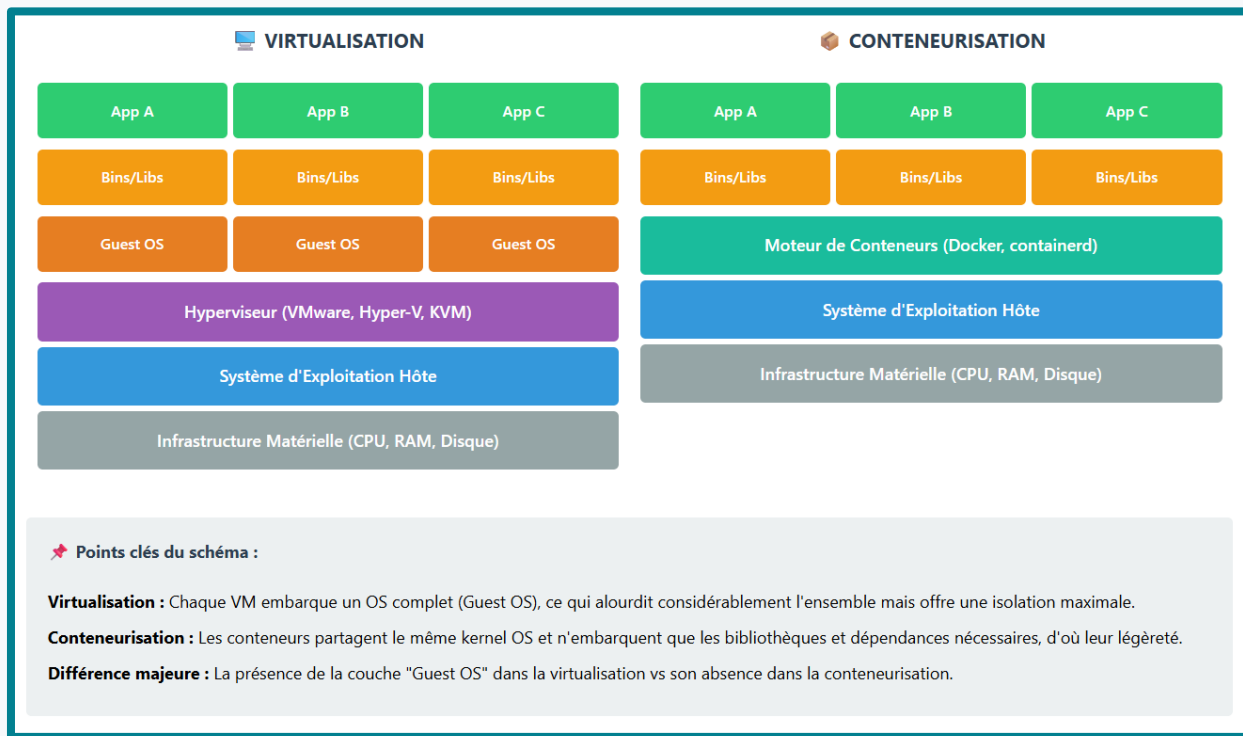
Performance

Pourquoi Docker ?

Pourquoi Docker ? Virtualisation et Conteneurisation

Pourquoi Docker ? Virtualisation et Conteneurisation

La différence essentielle est que la virtualisation crée des machines complètes avec leur propre OS, tandis que la conteneurisation isole juste les applications en partageant le même noyau système.



Pourquoi Docker ?

Problématique

Problèmes de scalabilité

Solutions apportées par Docker

Pourquoi Docker ?

Problématique

Problèmes de scalabilité

Solutions apportées par Docker

- Portabilité : environnements cohérents (conteneurs qui encapsulent une application et toutes ses dépendances dans un environnement isolé), indépendance de l'environnement, facilité de partage

Portabilité

Pourquoi Docker ?

Problématique

Problèmes de scalabilité

Solutions apportées par Docker

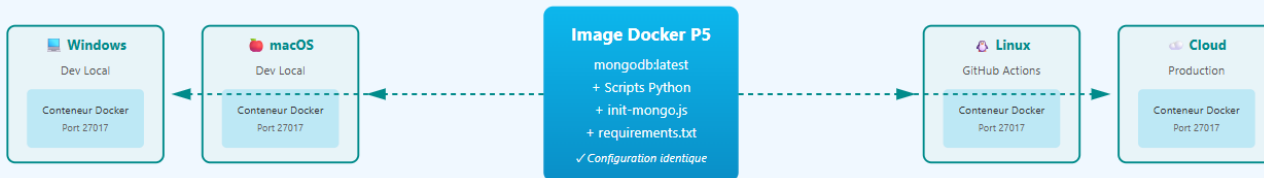
- Portabilité : environnements cohérents (conteneurs qui encapsulent une application et toutes ses dépendances dans un environnement isolé), indépendance de l'environnement, facilité de partage
- Facilité de déploiement : déploiement simplifié, automatisation, scalabilité, isolation et sécurité, gestion des dépendances

Portabilité

**Facilité de
déploiement**

Pourquoi Docker ?

PORTABILITÉ : Une image, tous les environnements

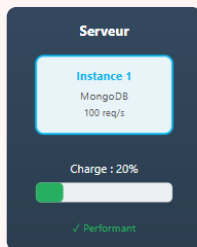


✓ Même image Docker = Comportement identique partout

"Ça fonctionne sur ma machine" → "Ça fonctionne partout !"

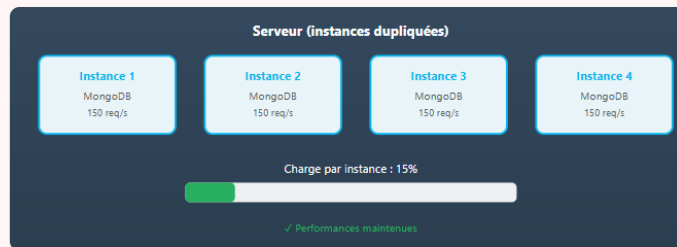
⚡ SCALABILITÉ : Adaptation automatique à la charge

📊 Charge faible



Charge ☒ →
+ 500 utilisateurs

🚀 Scaling automatique (docker-compose scale)



\$ docker-compose up --scale migration=4

→ Duplication instantanée de l'image en 4 instances isolées

Pourquoi AWS ?

Problématique

Problèmes de scalabilité

Solutions apportées par AWS

Pourquoi AWS ?

Problématique

Problèmes de scalabilité

Solutions apportées par AWS

- Fiabilité : infrastructure mondiale, engagement de disponibilité, sauvegardes et reprise après sinistre, sécurité

Fiabilité

Pourquoi AWS ?

Problématique

Problèmes de scalabilité

Solutions apportées par AWS

- Fiabilité : infrastructure mondiale, engagement de disponibilité, sauvegardes et reprise après sinistre, sécurité
- Services gérés : réduction de la charge opérationnelle, simplicité de déploiement et de gestion, intégration facile

Fiabilité

**Services
gérés**

Pourquoi AWS ?

Problématique

Problèmes de scalabilité

Solutions apportées par AWS

- Fiabilité : infrastructure mondiale, engagement de disponibilité, sauvegardes et reprise après sinistre, sécurité
- Services gérés : réduction de la charge opérationnelle, simplicité de déploiement et de gestion, intégration facile
- Scalabilité : élastiquement scalable, scalabilité horizontale et verticale, adaptation aux charges de travail variables, coût-efficacité

Fiabilité

**Services
gérés**

Scalabilité

Justification des Choix Techniques

- **MongoDB** : Flexibilité et scalabilité pour les données médicales.
- **Python et Pandas** : Traitement et analyse des données.
- **Docker Compose** : Portabilité et isolation des environnements .

Fonctionnalités

Détermination de l'environnement

transfert

optimisation

test intégrité

gestion des
utilisateurs



local
=> dév. + tests



Docker => test



Docker

*Utilisation de variables
d'environnements*

.env.local

.env.test_docker

.env.docker

Détermination de l'environnement

Variable ENVIRONMENT	Fichier .env chargé	Utilisation
"local"	.env.local	Développement en local
"test_docker"	.env.test_docker	Tests dans Docker (CI/CD)
"docker"	.env.docker	Production avec Docker

Shell	Commande
PowerShell	\$env:ENVIRONMENT = "local"
CMD (Windows)	set ENVIRONMENT=local
Bash (Linux/Mac)	export ENVIRONMENT=local

Détermination de l'environnement

```
6 # Détermine l'environnement
7 env = os.getenv("ENVIRONMENT", "local") # 'local' par défaut
8
9 # Charge le fichier .env approprié
10 if env == "local":
11     load_dotenv(".env.local")
12 elif env == "docker":
13     load_dotenv(".env.docker")
14 elif env == "test_docker":
15     load_dotenv(".env.test_docker")
16 else:
17     load_dotenv() # Fichier .env par défaut
```

```
21 # Utilisation des variables d'environnement ou des valeurs par défaut
22 mongo_uri = mongo_uri or os.getenv("MONGO_URI", "mongodb://root:example@localhost:27017/")
23 db_name = db_name or os.getenv("DB_NAME", "P5_test") # Défaut sécurisé : base de test
24 csv_path = csv_file_path or os.getenv("CSV_FILE_PATH", "./data/healthcare_dataset.csv")
25
26 print(f"Exécution en mode {env}")
27 print(f"Base de données: {db_name}")
28 print(f"MongoDB URI: {mongo_uri}")
```

Détermination de l'environnement

transfert_csv_mongodb.py

optimiser_collection.py

test_integrite_donnees.py

gestion_utilisateurs.py



```
6  # Détermine l'environnement
7  env = os.getenv("ENVIRONMENT", "local") # 'local' par défaut
8
9  # Charge le fichier .env approprié
10 if env == "local":
11     load_dotenv(".env.local")
12 elif env == "docker":
13     load_dotenv(".env.docker")
14 elif env == "test_docker":
15     load_dotenv(".env.test_docker")
16 else:
17     load_dotenv() # Fichier .env par défaut
```

```
21 # Utilisation des variables d'environnement ou des valeurs par défaut
22 mongo_uri = mongo_uri or os.getenv("MONGO_URI", "mongodb://root:example@localhost:27017/")
23 db_name = db_name or os.getenv("DB_NAME", "P5_test") # Défaut sécurisé : base de test
24 csv_path = csv_file_path or os.getenv("CSV_FILE_PATH", "./data/healthcare_dataset.csv")
25
26 print(f"Exécution en mode {env}")
27 print(f"Base de données: {db_name}")
28 print(f"MongoDB URI: {mongo_uri}")
```

Migration CSV → MongoDB

 transfert_csv_mongodb.py

Migration CSV → MongoDB

 transfert_csv_mongodb.py

.env.local

Migration CSV → MongoDB

 transfert_csv_mongodb.py

.env.local

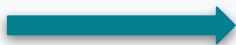
```
.env.local
1  # Environnement
2  ENVIRONMENT=local
3
4  # MongoDB - Compte root
5  MONGO_ROOT_USER=root
6  MONGO_ROOT_PASSWORD=exemple
7
8  # MongoDB - Mots de passe des utilisateurs RBAC (utilisés par init-mongo.js)
9  MONGO_ADMIN_PASSWORD=SecureAdminPassword123!
10 MONGO_LECTEUR_PASSWORD=SecureLecteurPassword123!
11 MONGO_REDACTEUR_PASSWORD=SecureRedacteurPassword123!
12 MONGO_ADMIN_TEST_PASSWORD=TestAdminPassword123!
13 MONGO_ADMIN_USER=admin_p5_test
14
15 # MongoDB - Connexion
16 MONGO_URI=mongodb://localhost:27017/
17
18 # Base de données
19 DB_NAME=P5_test
20
21 # Chemin CSV
22 CSV_FILE_PATH=./data/healthcare_dataset.csv
```

Migration CSV → MongoDB

 transfert_csv_mongodb.py



.env.local



transfert_csv_mongodb

```
.env.local
1  # Environnement
2  ENVIRONMENT=local
3
4  # MongoDB - Compte root
5  MONGO_ROOT_USER=root
6  MONGO_ROOT_PASSWORD=exemple
7
8  # MongoDB - Mots de passe des utilisateurs RBAC (utilisés par init-mongo.js)
9  MONGO_ADMIN_PASSWORD=SecureAdminPassword123!
10 MONGO_LECTEUR_PASSWORD=SecureLecteurPassword123!
11 MONGO_REDACTEUR_PASSWORD=SecureRedacteurPassword123!
12 MONGO_ADMIN_TEST_PASSWORD=TestAdminPassword123!
13 MONGO_ADMIN_USER=admin_p5_test
14
15 # MongoDB - Connexion
16 MONGO_URI=mongodb://localhost:27017/
17
18 # Base de données
19 DB_NAME=P5_test
20
21 # Chemin CSV
22 CSV_FILE_PATH=./data/healthcare_dataset.csv
```

Migration CSV → MongoDB

 transfert_csv_mongodb.py



.env.local

transfert_csv_mongodb



P5_test
MongoDB

```
.env.local
1 # Environnement
2 ENVIRONMENT=local
3
4 # MongoDB - Compte root
5 MONGO_ROOT_USER=root
6 MONGO_ROOT_PASSWORD=example
7
8 # MongoDB - Mots de passe des utilisateurs RBAC (utilisés par init-mongo.js)
9 MONGO_ADMIN_PASSWORD=SecureAdminPassword123!
10 MONGO_LECTEUR_PASSWORD=SecureLecteurPassword123!
11 MONGO_REDACTEUR_PASSWORD=SecureRedacteurPassword123!
12 MONGO_ADMIN_TEST_PASSWORD=TestAdminPassword123!
13 MONGO_ADMIN_USER=admin_p5_test
14
15 # MongoDB - Connexion
16 MONGO_URI=mongodb://localhost:27017/
17
18 # Base de données
19 DB_NAME=P5_test
20
21 # Chemin CSV
22 CSV_FILE_PATH=./data/healthcare_dataset.csv
```

Migration CSV → MongoDB

 transfert_csv_mongodb.py

.env.local



transfert_csv_mongodb



P5_test
MongoDB



```

.env.local
1 # Environnement
2 ENVIRONMENT=local
3
4 # MongoDB - Compte root
5 MONGO_ROOT_USER=root
6 MONGO_ROOT_PASSWORD=example
7
8 # MongoDB - Mots de passe des utilisateurs RBAC (utilisés par init-mongo.js)
9 MONGO_ADMIN_PASSWORD=SecureAdminPassword123!
10 MONGO_LECTEUR_PASSWORD=SecureLecteurPassword123!
11 MONGO_REDACTEUR_PASSWORD=SecureRedacteurPassword123!
12 MONGO_ADMIN_TEST_PASSWORD=TestAdminPassword123!
13 MONGO_ADMIN_USER=admin_p5_test
14
15 # MongoDB - Connexion
16 MONGO_URI=mongodb://localhost:27017/
17
18 # Base de données
19 DB_NAME=P5_test
20
21 # Chemin CSV
22 CSV_FILE_PATH=./data/healthcare_dataset.csv
```

```

• (.venv) PS C:\Users\moymo\OC\P5> $env:ENVIRONMENT="local"
• (.venv) PS C:\Users\moymo\OC\P5> python scripts/transfert_csv_mongodb.py
Début de l'exécution de transfert_csv_mongodb.py
Exécution en mode local
Base de données: P5_test
MongoDB URI: mongodb://localhost:27017/
Fichier CSV trouvé: ./data/healthcare_dataset.csv
Environnement de test détecté. Nettoyage de la base 'P5_test'...
Collection 'dataset_donnees_medicales' vidée.
55500 documents insérés dans la collection 'dataset_donnees_medicales' de la base 'P5_test'.
Fin de l'exécution de transfert_csv_mongodb.py
```

Normalisation + index

 `optimiser_collection.py`

Normalisation + index

 optimiser_collection.py

Normalisation des noms

Normalisation + index

 optimiser_collection.py

Normalisation des noms

Name : "andrEw waTtS"



Name : "Andrew Watts"

Normalisation + index

 optimiser_collection.py

Normalisation des noms

Name : "andrEw waTtS"



Name : "Andrew Watts"

Normalisation des dates

Normalisation + index

 optimiser_collection.py

Normalisation des noms

Name : "andrEw waTtS"



Name : "Andrew Watts"

Normalisation des dates

Discharge Date : "2020-12-18"

Date of Admission : "2020-11-18"



Date of Admission : 2020-11-18T00:00:00.000+00:00

Discharge Date : 2020-12-18T00:00:00.000+00:00

Conversion du format string au type Date natif MongoDB pour permettre des requêtes temporelles efficaces

Normalisation + index

 optimiser_collection.py

Création d'index

Normalisation + index

 optimiser_collection.py

Création d'index

Performances de recherche

Tri optimisé

Agrégations performantes

Recherche Full-Text

Utilisation mémoire réduite

Normalisation + index

 optimiser_collection.py

Création d'index

Performances de recherche

Tri optimisé

Agrégations performantes

Recherche Full-Text

Utilisation mémoire réduite

Espace disque supplémentaire (~10-20% de la taille des données)

Ralentissement léger des insertions/modifications

Normalisation + index

 optimiser_collection.py

Création d'index

Performances de recherche

Tri optimisé

Agrégations performantes

Recherche Full-Text

Utilisation mémoire réduite

Espace disque supplémentaire (~10-20% de la taille des données)

Ralentissement léger des insertions/modifications

Index	Champ(s)	Justification
Name_1	Name	Recherche par patient
Medical_Condition_1	Medical Condition	Filtrage par pathologie
Date_of_Admission_1	Date of Admission	Tri et filtrage temporel
Medical_Condition_1_Age	Medical Condition + Age	analytiques combinées
text_index	Tous les champs	Recherche textuelle libre

Normalisation + index

 optimiser_collection.py

```
(.venv) PS C:\Users\moymo\OC\P5> $env:ENVIRONMENT
local
(.venv) PS C:\Users\moymo\OC\P5> python scripts/optimiser_collection.py
Début de l'exécution de optimiser_collection.py
Exécution en mode local
Base de données: P5_test
MongoDB URI: mongodb://localhost:27017/
Optimisation de la collection 'dataset-donnees-medicales' dans la base 'P5_test'...
Suppression des index existants...
Normalisation des noms et prénoms...
Mise à jour des noms et prénoms terminée. 0 documents mis à jour.
Conversion des dates d'admission...
Conversion des dates de sortie...
Création des index...
Index 'Name_1' créé sur le champ 'Name'
Index 'Medical_Condition_1' créé sur le champ 'Medical Condition'
Index 'Date_of_Admission_1' créé sur le champ 'Date of Admission'
Index 'Medical_Condition_1_Age_1' créé sur ('Medical Condition', 'Age')
Index 'text_index' créé sur tous les champs

Index disponibles dans la collection :
- _id_: [('id', 1)]
- Name_1: [('Name', 1)]
- Medical_Condition_1: [('Medical Condition', 1)]
- Date_of_Admission_1: [('Date of Admission', 1)]
- Medical_Condition_1_Age_1: [('Medical Condition', 1), ('Age', 1)]
- text_index: [('fts', 'text'), ('ftsx', 1)]

Optimisation de la collection terminée avec succès.
Fin de l'exécution de optimiser_collection.py
```

Normalisation + index

optimiser_collection.py

```
PS C:\Users\moymo\OC\PS> mongosh "mongodb://root:example@localhost:27017/P5_test" --authenticationDatabase admin
>>
Current Mongosh Log ID: 699c64e470c848c12f628c9f
Connecting to:  mongodb://<credentials>@localhost:27017/P5_test?directConnection=true&serverSelectionTimeoutMS=2000&authSource=admin&appName=mongosh+2.6.0
Using MongoDB:  8.2.3
Using Mongosh:  2.6.0
```

```
PS_test> db.dataset_donnees_medicales.find({ "Medical Condition": "Diabetes" }).explain("executionStats")
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'P5_test.dataset_donnees_medicales',
    parsedQuery: { 'Medical Condition': { '$eq': 'Diabetes' } },
    indexFilterSet: false,
    queryHash: '03307585',
    planCacheShapeHash: '03307585',
    planCacheKey: '31633950',
    optimizationTimeMillis: 12,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'FETCH',
      inputStage: {
        stage: 'IXSCAN',
        keyPattern: { 'Medical Condition': 1 },
        indexName: 'Medical_Condition_1',
        isMultiKey: false,
        multiKeyPaths: { 'Medical Condition': [ ] },
        isUnique: false,
        isSparse: false,
        isPartial: false,
        indexVersion: 2,
        direction: 'forward',
        indexBounds: { 'Medical Condition': [ '['Diabetes', 'Diabetes']' ] }
      }
    }
  },
}
```

Confirmation de l'utilisation de l'index

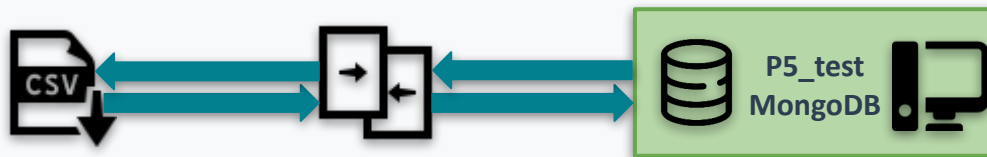
Vérification intégrité

 test_integrite_donnees.py

Vérification intégrité

`test_integrite_donnees.py`

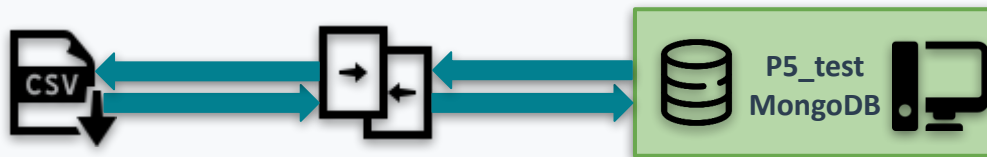
va comparer les données entre le fichier csv et la base de données crée.



Vérification intégrité

`test_integrite_donnees.py`

va comparer les données entre le fichier csv et la base de données crée.



Type de données

Doublons

Valeurs manquantes

Nombre de lignes

Index

Vérification intégrité

test_integrite_donnees.py

```
• (.venv) PS C:\Users\moymo\OC\P5> python scripts/test_integrite_donnees.py
Début de l'exécution de test_integrite_donnees.py
Exécution en mode local
Base de données: P5_test
MongoDB URI: mongodb://localhost:27017/
Fichier CSV trouvé: ./data/healthcare_dataset.csv
```

Vérification intégrité

test_integrite_donnees.py

```
(.venv) PS C:\Users\moymo\OC\P5> python scripts/test_integrite_donnees.py
Début de l'exécution de test_integrite_donnees.py
Exécution en mode local
Base de données: P5_test
MongoDB URI: mongodb://localhost:27017/
Fichier CSV trouvé: ./data/healthcare_dataset.csv
```

Type de données

```
=== Vérification des colonnes ===
Différence dans les colonnes : set()

=== Vérification des types de données ===
          CSV          MongoDB
Admission Type  object    object
Age             int64     int64
Billing Amount  float64    float64
Blood Type      object    object
Date of Admission object  datetime64[ns]
Discharge Date  object  datetime64[ns]
Doctor          object    object
Gender          object    object
Hospital        object    object
Insurance Provider object  object
Medical Condition object  object
Medication      object    object
Name            object    object
Room Number     int64     int64
Test Results    object    object
_id             NaN       object
```

Vérification intégrité

test_integrite_donnees.py

```
• (.venv) PS C:\Users\moymo\OC\P5> python scripts/test_integrite_donnees.py
Début de l'exécution de test_integrite_donnees.py
Exécution en mode local
Base de données: P5_test
MongoDB URI: mongodb://localhost:27017/
Fichier CSV trouvé: ./data/healthcare_dataset.csv
```

Type de données

```
=== Vérification des colonnes ===
Différence dans les colonnes : set()

=== Vérification des types de données ===
          CSV      MongoDB
Admission Type  object  object
Age             int64   int64
Billing Amount  float64  float64
Blood Type      object  object
Date of Admission  object  datetime64[ns]
Discharge Date  object  datetime64[ns]
Doctor          object  object
Gender          object  object
Hospital        object  object
Insurance Provider  object  object
Medical Condition  object  object
Medication      object  object
Name            object  object
Room Number     int64   int64
Test Results    object  object
_id             NaN     object
```

Doublons

```
=== Vérification des doublons ===
Nombre de doublons dans le CSV : 534
Nombre de doublons dans MongoDB : 0
```

Vérification intégrité

 test_integrite_donnees.py

Valeurs manquantes

```
=== Vérification des valeurs manquantes ===  
Valeurs manquantes dans le CSV :  
Name          0  
Age           0  
Gender        0  
Blood Type    0  
Medical Condition 0  
Date of Admission 0  
Doctor        0  
Hospital      0  
Insurance Provider 0  
Billing Amount 0  
Room Number   0  
Admission Type 0  
Discharge Date 0  
Medication    0  
Test Results  0  
dtype: int64  
  
Valeurs manquantes dans MongoDB :  
_id          0  
Name        0  
Age         0  
Gender      0  
Blood Type  0  
Medical Condition 0  
Blood Type  0  
Blood Type  0  
Blood Type  0  
Blood Type  0  
Medical Condition 0  
Date of Admission 0  
Billing Amount 0  
Room Number   0  
Admission Type 0  
Discharge Date 0  
Medication    0  
Test Results  0  
dtype: int64
```

Vérification intégrité

 test_integrite_donnees.py

Valeurs manquantes

```
=== Vérification des valeurs manquantes ===
Valeurs manquantes dans le CSV :
Name          0
Age           0
Gender        0
Blood Type    0
Medical Condition 0
Date of Admission 0
Doctor        0
Hospital      0
Insurance Provider 0
Billing Amount 0
Room Number   0
Admission Type 0
Discharge Date 0
Medication    0
Test Results  0
dtype: int64

Valeurs manquantes dans MongoDB :
_id          0
Name         0
Age          0
Gender       0
Blood Type   0
Medical Condition 0
Blood Type   0
Blood Type   0
Blood Type   0
Blood Type   0
Medical Condition 0
Date of Admission 0
Billing Amount 0
Room Number   0
Admission Type 0
Discharge Date 0
Medication    0
Test Results  0
dtype: int64
```

Nombre de lignes

```
=== Vérification du nombre de lignes ===
Nombre de lignes dans le CSV : 55500
Nombre de lignes dans MongoDB : 55500
```


Vérification intégrité

 test_integrite_donnees.py

Valeurs manquantes

```
=== Vérification des valeurs manquantes ===
Valeurs manquantes dans le CSV :
Name          0
Age           0
Gender        0
Blood Type    0
Medical Condition 0
Date of Admission 0
Doctor        0
Hospital      0
Insurance Provider 0
Billing Amount 0
Room Number   0
Admission Type 0
Discharge Date 0
Medication    0
Test Results  0
dtype: int64

Valeurs manquantes dans MongoDB :
_id          0
Name         0
Age          0
Gender       0
Blood Type   0
Medical Condition 0
Blood Type   0
Blood Type   0
Blood Type   0
Blood Type   0
Medical Condition 0
Date of Admission 0
Billing Amount 0
Room Number   0
Admission Type 0
Discharge Date 0
Medication    0
Test Results  0
dtype: int64
```

Nombre de lignes

```
=== Vérification du nombre de lignes ===
Nombre de lignes dans le CSV : 55500
Nombre de lignes dans MongoDB : 55500
```

Index

```
=== Vérification des index ===
Succès : Tous les index requis ont été créés.
Fin de l'exécution de test_integrite_donnees.py
```

Création d'utilisateurs applicatifs

 gestion_utilisateurs.py

Création d'utilisateurs applicatifs

 gestion_utilisateurs.py

Définition de rôles applicatifs

Création d'utilisateurs applicatifs

 gestion_utilisateurs.py

Définition de rôles applicatifs

```
25 # Rôles applicatifs disponibles
26 ROLES = {
27     "admin": {
28         "description": "Administrateur système - tous les droits",
29         "permissions": ["read", "write", "delete", "manage_users"]
30     },
31     "medecin": {
32         "description": "Médecin - lecture et écriture des dossiers médicaux",
33         "permissions": ["read", "write"]
34     },
35     "infirmier": {
36         "description": "Infirmier - lecture et écriture limitée",
37         "permissions": ["read", "write_limited"]
38     },
39     "lecteur": {
40         "description": "Lecteur - consultation uniquement",
41         "permissions": ["read"]
42     }
43 }
```

Création d'utilisateurs applicatifs

 gestion_utilisateurs.py

```
def valider_mot_de_passe(password):
```

```
def hasher_mot_de_passe(password):
```

```
def verifier_mot_de_passe(password, hashed_password):
```

```
def creer_utilisateur(username, password, role, email=None, db=None, mongo_uri=None, db_name=None):
```

```
def authentifier_utilisateur(username, password, mongo_uri=None, db_name=None):
```

```
def lister_utilisateurs(mongo_uri=None, db_name=None):
```

```
def supprimer_utilisateur(username, mongo_uri=None, db_name=None):
```

Sécurité RBAC

Sécurité RBAC

Le projet implémente une **architecture de sécurité à deux niveaux**.

Sécurité RBAC

Le projet implémente une **architecture de sécurité à deux niveaux**.

RBAC MongoDB (Niveau infrastructure)

Contrôle d'accès au niveau de la base de données elle-même.

Compte MongoDB	Rôle	Permissions	Base
root	Super admin	Tout	Toutes
admin_p5	adminP5	CRUD + gestion collections/index	P5
lecteur_p5	lecteurMedical	Lecture seule	
redacteur_p5	redacteurMedical	Lecture + Écriture (insert, update)	P5
admin_p5_test	adminP5	CRUD + gestion collections/index	P5_test

Sécurité RBAC

Le projet implémente une **architecture de sécurité à deux niveaux**.

RBAC MongoDB (Niveau infrastructure)

Contrôle d'accès au niveau de la base de données elle-même.

Compte MongoDB	Rôle	Permissions	Base
root	Super admin	Tout	Toutes
admin_p5	adminP5	CRUD + gestion collections/index	P5
lecteur_p5	lecteurMedical	Lecture seule	
redacteur_p5	redacteurMedical	Lecture + Écriture (insert, update)	P5
admin_p5_test	adminP5	CRUD + gestion collections/index	P5_test

RBAC Applicatif (Niveau application)

Contrôle d'accès métier pour les utilisateurs finaux.

Rôle applicatif	Permissions métier
admin	read, write, delete, manage_users
medecin	read, write
infirmier	read, write_limited
lecteur	read

Sécurité des mots de passe

Sécurité des mots de passe

- Hashing bcrypt avec 12 rounds (résistant GPU)
- Salt automatique unique par utilisateur
- Validation stricte : 12+ caractères, majuscule, minuscule, chiffre, spécial
- Soft delete : désactivation au lieu de suppression

Étape 1 : Développement & Tests unitaires

Environnement : Local Python + venv

MongoDB : Non requis (mocks)

Étape 2 : Tests d'intégration locaux

Environnement : Local Python + MongoDB local

MongoDB : Requis (localhost:27017)

Étape 3 : Validation Docker (tests)

Environnement : Docker test (port 27018)

MongoDB : Docker mongodb_test

Étape 4 : Déploiement production

Environnement : Docker production (port 27017)

MongoDB : Docker mongodb

Développement & Tests unitaires

Développement & Tests unitaires

Environnement : Local Python + venv

MongoDB : Non requis (mocks)

Développement & Tests unitaires

Environnement : Local Python + venv

MongoDB : Non requis (mocks)

Pytest = Outil pour automatiser les tests

Mock = Faux objet pour tester sans dépendances externes

Tests unitaires avec mocks ⇒ **test rapide de la logique métier des scripts sans avoir besoin de MongoDB**

Développement & Tests unitaires

Environnement : Local Python + venv

MongoDB : Non requis (mocks)

Pytest = Outil pour automatiser les tests

Mock = Faux objet pour tester sans dépendances externes

Tests unitaires avec mocks ⇒ **test rapide de la logique métier des scripts sans avoir besoin de MongoDB**

 `test_transfert_csv_mongodb.py`

 `test_optimiser_collection.py`

 `test_test_integrite_donnees.py`

 `test_gestion_utilisateurs.py`

Développement & Tests unitaires

Environnement : Local Python + venv

MongoDB : Non requis (mocks)

Pytest = Outil pour automatiser les tests

Mock = Faux objet pour tester sans dépendances externes

Tests unitaires avec mocks ⇒ **test rapide de la logique métier des scripts sans avoir besoin de MongoDB**

test_transfert_csv_mongodb.py

test_optimiser_collection.py

test_test_integrite_donnees.py

test_gestion_utilisateurs.py

24 tests unitaires avec mocks

Développement & Tests unitaires

```
• (.venv) PS C:\Users\moymo\OC\P5> pytest tests/test_transfert_csv_mongodb.py -v
===== test session starts =====
platform win32 -- Python 3.14.0, pytest-7.4.0, pluggy-1.6.0 -- C:\Users\moymo\OC\P5\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\moymo\OC\P5
configfile: pytest.ini
collected 1 item

tests/test_transfert_csv_mongodb.py::test_transfert_csv PASSED [100%]

===== 1 passed in 0.03s =====
• (.venv) PS C:\Users\moymo\OC\P5> pytest tests/test_optimiser_collection.py -v
===== test session starts =====
platform win32 -- Python 3.14.0, pytest-7.4.0, pluggy-1.6.0 -- C:\Users\moymo\OC\P5\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\moymo\OC\P5
configfile: pytest.ini
collected 1 item

tests/test_optimiser_collection.py::test_optimiser_collection PASSED [100%]

===== 1 passed in 0.03s =====
• (.venv) PS C:\Users\moymo\OC\P5> pytest tests/test_test_integrite_donnees.py -v
===== test session starts =====
platform win32 -- Python 3.14.0, pytest-7.4.0, pluggy-1.6.0 -- C:\Users\moymo\OC\P5\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\moymo\OC\P5
configfile: pytest.ini
collected 1 item

tests/test_test_integrite_donnees.py::test_integrite_donnees PASSED [100%]

===== 1 passed in 0.04s =====
```

Développement & Tests unitaires

```
(.env) PS C:\Users\moymo\OC\PS> pytest tests/test_gestion_utilisateurs.py -v
===== test session starts =====
platform win32 -- Python 3.14.0, pytest-7.4.0, pluggy-1.6.0 -- C:\Users\moymo\OC\PS\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\moymo\OC\PS
configfile: pytest.ini
collected 21 items

tests/test_gestion_utilisateurs.py::TestValidationMotDePasse::test_mot_de_passe_trop_court PASSED [ 4%]
tests/test_gestion_utilisateurs.py::TestValidationMotDePasse::test_mot_de_passe_sans_majuscule PASSED [ 9%]
tests/test_gestion_utilisateurs.py::TestValidationMotDePasse::test_mot_de_passe_sans_minuscule PASSED [ 14%]
tests/test_gestion_utilisateurs.py::TestValidationMotDePasse::test_mot_de_passe_sans_chiffre PASSED [ 19%]
tests/test_gestion_utilisateurs.py::TestValidationMotDePasse::test_mot_de_passe_sans_caractere_special PASSED [ 23%]
tests/test_gestion_utilisateurs.py::TestValidationMotDePasse::test_mot_de_passe_valide PASSED [ 28%]
tests/test_gestion_utilisateurs.py::TestHashageMotDePasse::test_hash_genere_different_a_chaque_fois PASSED [ 33%]
tests/test_gestion_utilisateurs.py::TestHashageMotDePasse::test_verification_mot_de_passe_correct PASSED [ 38%]
tests/test_gestion_utilisateurs.py::TestHashageMotDePasse::test_verification_mot_de_passe_incorrect PASSED [ 42%]
tests/test_gestion_utilisateurs.py::TestHashageMotDePasse::test_hash_commence_par_bcrypt_prefix PASSED [ 47%]
tests/test_gestion_utilisateurs.py::TestCreationUtilisateur::test_creation_utilisateur_valide PASSED [ 52%]
tests/test_gestion_utilisateurs.py::TestCreationUtilisateur::test_creation_utilisateur_existant PASSED [ 57%]
tests/test_gestion_utilisateurs.py::TestCreationUtilisateur::test_creation_utilisateur_role_invalide PASSED [ 61%]
tests/test_gestion_utilisateurs.py::TestCreationUtilisateur::test_creation_utilisateur_mot_de_passe_faible PASSED [ 66%]
tests/test_gestion_utilisateurs.py::TestAuthentification::test_authentification_reussie PASSED [ 71%]
tests/test_gestion_utilisateurs.py::TestAuthentification::test_authentification_mot_de_passe_incorrect PASSED [ 76%]
tests/test_gestion_utilisateurs.py::TestAuthentification::test_authentification_utilisateur_inexistent PASSED [ 80%]
tests/test_gestion_utilisateurs.py::TestAuthentification::test_authentification_compte_desactive PASSED [ 85%]
tests/test_gestion_utilisateurs.py::TestRoles::test_roles_definis PASSED [ 90%]
tests/test_gestion_utilisateurs.py::TestRoles::test_roles_ont_permissions PASSED [ 95%]
tests/test_gestion_utilisateurs.py::TestRoles::test_admin_a_toutes_permissions PASSED [100%]

===== 21 passed in 2.71s =====
```

Étape 1 : Développement & Tests unitaires

Environnement : Local Python + venv

MongoDB : Non requis (mocks)

Étape 2 : Tests d'intégration locaux

Environnement : Local Python + MongoDB local

MongoDB : Requis (localhost:27017)

Étape 3 : Validation Docker (tests)

Environnement : Docker test (port 27018)

MongoDB : Docker mongodb_test

Étape 4 : Déploiement production

Environnement : Docker production (port 27017)

MongoDB : Docker mongodb

Tests d'intégration locaux

Tests d'intégration locaux

Environnement : Local Python + MongoDB local

MongoDB : Requis (localhost:27017)

Tests d'intégration locaux

Environnement : Local Python + MongoDB local

MongoDB : Requis (localhost:27017)

Tests d'intégration ⇒ test du système complet avec une vraie connexion MongoDB

Tests d'intégration locaux

Environnement : Local Python + MongoDB local

MongoDB : Requis (localhost:27017)

Tests d'intégration ⇒ test du système complet avec une vraie connexion MongoDB

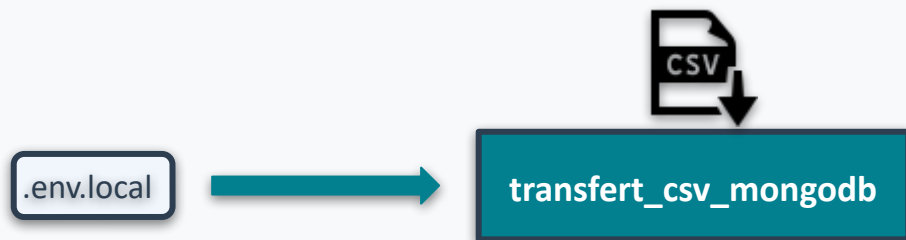
.env.local

Tests d'intégration locaux

Environnement : Local Python + MongoDB local

MongoDB : Requis (localhost:27017)

Tests d'intégration ⇒ **test du système complet avec une vraie connexion MongoDB**



Tests d'intégration locaux

Environnement : Local Python + MongoDB local

MongoDB : Requis (localhost:27017)

Tests d'intégration ⇒ test du système complet avec une vraie connexion MongoDB



Tests d'intégration locaux

Environnement : Local Python + MongoDB local

MongoDB : Requis (localhost:27017)

Tests d'intégration ⇒ **test du système complet avec une vraie connexion MongoDB**

①

Migration et optimisation

transfert_csv_mongodb.py

optimiser_collection.py

test_integrite_donnees.py

gestion_utilisateurs.py

Tests d'intégration locaux

Environnement : Local Python + MongoDB local

MongoDB : Requis (localhost:27017)

Tests d'intégration ⇒ test du système complet avec une vraie connexion MongoDB

①

Migration et optimisation

transfert_csv_mongodb.py

optimiser_collection.py

test_integrite_donnees.py

gestion_utilisateurs.py

②

Tests RBAC

test_rbac_integration.py



4 tests d'intégration

Tests d'intégration locaux

Environnement : Local Python + MongoDB local

MongoDB : Requis (localhost:27017)

Tests d'intégration ⇒ test du système complet avec une vraie connexion MongoDB

①

Migration et optimisation

transfert_csv_mongodb.py

optimiser_collection.py

test_integrite_donnees.py

gestion_utilisateurs.py

- 55 500 documents dans MongoDB
- 5 index créés
- Données normalisées et validées

②

Tests RBAC

test_rbac_integration.py



4 tests d'intégration

- Connexion à MongoDB
- Création d'utilisateurs applicatifs
- Authentification avec bcrypt
- Validation des rôles

Tests d'intégration locaux

Environnement : Local Python + MongoDB local

MongoDB : Requis (localhost:27017)

Tests d'intégration ⇒ test du système complet avec une vraie connexion MongoDB

```
• (.venv) PS C:\Users\moymo\OC\PS> pytest tests/test_rbac_integration.py -v
===== test session starts =====
platform win32 -- Python 3.14.0, pytest-7.4.0, pluggy-1.6.0 -- C:\Users\moymo\OC\PS\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\moymo\OC\PS
configfile: pytest.ini
collected 4 items

tests/test_rbac_integration.py::TestRBACIntegration::test_connexion_mongodb PASSED [ 25%]
tests/test_rbac_integration.py::TestRBACIntegration::test_creation_utilisateur_succes PASSED [ 50%]
tests/test_rbac_integration.py::TestRBACIntegration::test_authentification_utilisateur PASSED [ 75%]
tests/test_rbac_integration.py::TestRBACIntegration::test_creation_utilisateur_role_invalide PASSED [100%]

===== 4 passed in 0.76s =====
```

Étape 1 : Développement & Tests unitaires

Environnement : Local Python + venv

MongoDB : Non requis (mocks)

Étape 2 : Tests d'intégration locaux

Environnement : Local Python + MongoDB local

MongoDB : Requis (localhost:27017)

Étape 3 : Validation Docker (tests)

Environnement : Docker test (port 27018)

MongoDB : Docker mongodb_test

Étape 4 : Déploiement production

Environnement : Docker production (port 27017)

MongoDB : Docker mongodb

Validation Docker (tests)

Validation Docker (tests)

Environnement : Docker test (port 27018)

MongoDB : Docker mongodb_test

Validation Docker (tests)

Environnement : Docker test (port 27018)

MongoDB : Docker mongodb_test

.env.test_docker

Validation Docker (tests)

Environnement : Docker test (port 27018)

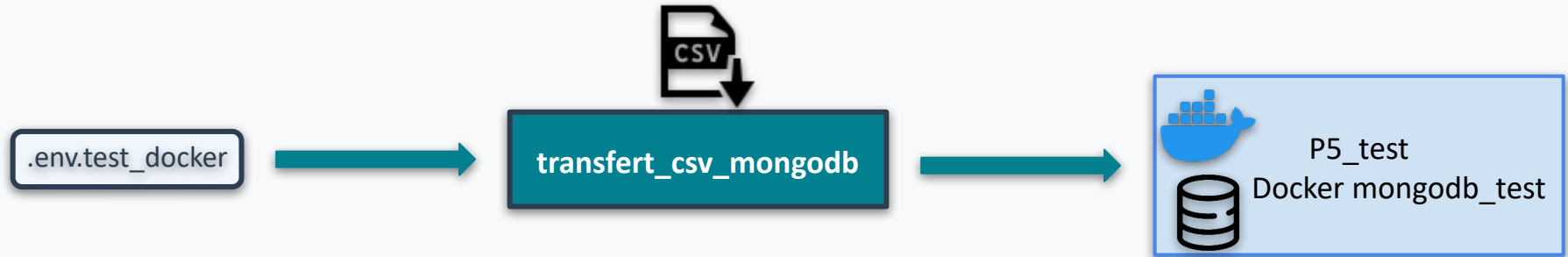
MongoDB : Docker mongodb_test



Validation Docker (tests)

Environnement : Docker test (port 27018)

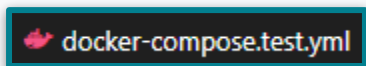
MongoDB : Docker mongodb_test



Validation Docker (tests)

Environnement : Docker test (port 27018)

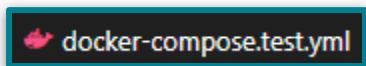
MongoDB : Docker mongodb_test



Validation Docker (tests)

Environnement : Docker test (port 27018)

MongoDB : Docker mongodb_test

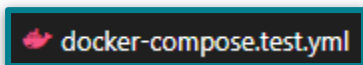


- lancer **plusieurs conteneurs** en une seule commande
- définir les **services** (web, db, cache, etc.)
- gérer :
 - ports
 - volumes
 - réseaux
 - variables d'environnement
 - dépendances entre services

Validation Docker (tests)

Environnement : Docker test (port 27018)

MongoDB : Docker mongodb_test



Simuler GitHub Actions

Isolation complète / Tester dans un environnement VIERGE

Validation multi-environnements / Tester la séquence COMPLÈTE

Validation Docker (tests)

Environnement : Docker test (port 27018)

MongoDB : Docker mongodb_test

Simuler GitHub Actions

Isolation complète / Tester dans un environnement VIERGE
Validation multi-environnements / Tester la séquence COMPLÈTE

 docker-compose.test.yml



mongodb_test



mongo:latest



test_runner:



python:3.11-slim

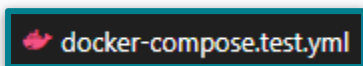
Validation Docker (tests)

Environnement : Docker test (port 27018)

MongoDB : Docker mongodb_test

Simuler GitHub Actions

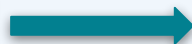
Isolation complète / Tester dans un environnement VIERGE
Validation multi-environnements / Tester la séquence COMPLÈTE



`mongodb_test`



mongo:latest



MongoDB



`test_runner`



python:3.11-slim



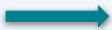
Réplication exacte du fichier
`run_tests.yml` de GitHub Actions

*mêmes étapes et mêmes
variables d'environnement*

Validation Docker (tests)

Environnement : Docker test (port 27018)

MongoDB : Docker mongodb_test



**Réplication exacte du fichier
run_tests.yml de GitHub Actions**

*mêmes étapes et mêmes
variables d'environnement*

Validation Docker (tests)

Environnement : Docker test (port 27018)

MongoDB : Docker mongodb_test



Workflow automatique:

1. MongoDB (port 27018) + init RBAC
2. Installation Python + dépendances
3. Tests unitaires (24 tests)
4. Tests intégration RBAC (4 tests)
5. Migration (55 500 documents)
6. Optimisation (5 index)
7. Validation intégrité

Validation Docker (tests)

Environnement : Docker test (port 27018)

MongoDB : Docker mongodb_test



Ordre stratégique : La sécurité RBAC est validée avant la migration des données sensibles

Workflow automatique:

1. MongoDB (port 27018) + init RBAC
2. Installation Python + dépendances
3. Tests unitaires (24 tests)
4. Tests intégration RBAC (4 tests)
5. Migration (55 500 documents)
6. Optimisation (5 index)
7. Validation intégrité

```

1  docker-compose.test.yml
2  services:
3      mongodb_test:
4          image: mongo:latest
5          container_name: mongodb_test
6          restart: unless-stopped
7          env_file:
8              - .env.test_docker
9          ports:
10             - "27018:27017"
11          volumes:
12             - mongodb_test_data:/data/db
13             - ./init-mongo.js:/docker-entrypoint-initdb.d/init-mongo.js:ro
14             - ./mongod.conf:/etc/mongod.conf:ro
15          networks:
16             - test_network
17          healthcheck:
18             test: echo 'db.runCommand("ping").ok' | mongosh mongodb_test:27017/test --quiet
19             interval: 10s
20             timeout: 10s
21             retries: 5
22             command: ["mongod", "--config", "/etc/mongod.conf"]
23
24  test_runner:
25      image: python:3.11-slim
26      container_name: test_runner
27      env_file:
28          - .env.test_docker
29      volumes:
30          - ./app
31      working_dir: /app
32      command: >
33          bash -c "set -e &&
34          echo '=== Installation des dépendances ===' &&
35          pip install --no-cache-dir -r requirements.txt &&
36          pip install pytest &&
37          echo '' &&
38          echo 'En attente de MongoDB...' &&
39          until timeout 1s bash -c 'cat < /dev/null > /dev/tcp/mongodb_test/27017' 2>/dev/null; do
40              sleep 1
41          done &&
42          echo 'MongoDB est prêt !' &&
43          echo '=== Tests unitaires ===' &&
44          python -m pytest tests/test_transfert_csv_mongodb.py -v &&
45          python -m pytest tests/test_optimiser_collection.py -v &&
46          python -m pytest tests/test_test_integrite_donnees.py -v &&
47          python -m pytest tests/test_gestion_utilisateurs.py -v &&
48          echo '' &&"

```

```

48      echo '=== Tests intégration - Sécurité RBAC ===' &&
49      pytest tests/test_rbac_integration.py -v &&
50      echo '' &&
51      echo '=== Tests intégration - Migration des données ===' &&
52      python scripts/transfert_csv_mongodb.py &&
53      python scripts/optimiser_collection.py &&
54      python scripts/test_integrite_donnees.py &&
55      echo '' &&
56      echo 'Tous les tests ont réussi !'
57
58  depends_on:
59      mongodb_test:
60          condition: service_healthy
61
62  networks:
63      - test_network
64
65  volumes:
66      mongodb_test_data:
67
68  networks:
69      test_network:
70          driver: bridge

```

```

1  docker-compose.test.yml
2  Run All Services
3  services:
4  Run Service
5  mongodb_test:
6  image: mongo:latest
7  container_name: mongodb_test
8  restart: unless-stopped
9  env_file:
10 - .env.test_docker
11 ports:
12 - "27018:27017"
13 volumes:
14 - mongodb_test_data:/data/db
15 - ./init-mongo.js:/docker-entrypoint-initdb.d/init-mongo.js:ro
16 - ./mongod.conf:/etc/mongod.conf:ro
17 networks:
18 - test_network
19 healthcheck:
20 test: echo 'db.runCommand("ping").ok' | mongosh mongodb_test:27017/test --quiet
21 interval: 10s
22 timeout: 10s
23 retries: 5
24 command: ["mongod", "--config", "/etc/mongod.conf"]
25
26 Run Service
27 test_runner:
28 image: python:3.11-slim
29 container_name: test_runner
30 env_file:
31 - .env.test_docker
32 volumes:
33 - ./app
34 working_dir: /app
35 command: >
36 bash -c "set -e &&
37 echo '=== Installation des dépendances ===' &&
38 pip install --no-cache-dir -r requirements.txt &&
39 pip install pytest &&
40 echo '' &&
41 echo 'En attente de MongoDB...' &&
42 until timeout 1s bash -c 'cat < /dev/null > /dev/tcp/mongodb_test/27017' 2>/dev/null; do
43 | sleep 1
44 done &&
45 echo 'MongoDB est prêt !' &&
46 echo '=== Tests unitaires ===' &&
47 python -m pytest tests/test_transfert_csv_mongodb.py -v &&
48 python -m pytest tests/test_optimiser_collection.py -v &&
49 python -m pytest tests/test_test_integrite_donnees.py -v &&
50 python -m pytest tests/test_gestion_utilisateurs.py -v &&
51 echo '' &&"

```

```

48 echo '=== Tests intégration - Sécurité RBAC ===' &&
49 pytest tests/test_rbac_integration.py -v &&
50 echo '' &&
51 echo '=== Tests intégration - Migration des données ===' &&
52 python scripts/transfert_csv_mongodb.py &&
53 python scripts/optimiser_collection.py &&
54 python scripts/test_integrite_donnees.py &&
55 echo '' &&
56 echo 'Tous les tests ont réussi !'
57 depends_on:
58   mongodb_test:
59     condition: service_healthy
60 networks:
61   - test_network
62
63 volumes:
64   mongodb_test_data:
65
66 networks:
67   test_network:
68     driver: bridge

```

```

docker-compose.test.yml
  Run All Services
1  services:
  Run Service
2  mongodb_test:
3    image: mongo:latest
4    container_name: mongodb_test
5    restart: unless-stopped
6    env_file:
7      - .env.test_docker
8    ports:
9      - "27018:27017"
10   volumes:
11     - mongodb_test_data:/data/db
12     - ./init-mongo.js:/docker-entrypoint-initdb.d/init-mongo.js:ro
13     - ./mongod.conf:/etc/mongod.conf:ro
14   networks:
15     - test_network
16   healthcheck:
17     test: echo 'db.runCommand("ping").ok' | mongosh mongodb_test:27017/test --quiet
18     interval: 10s
19     timeout: 10s
20     retries: 5
21   command: ["mongod", "--config", "/etc/mongod.conf"]
22
  Run Service
23  test_runner:
24    image: python:3.11-slim
25    container_name: test_runner
26    env_file:
27      - .env.test_docker
28    volumes:
29      - ./app
30    working_dir: /app
31    command: >
32      bash -c "set -e &&
33      echo '=== Installation des dépendances ===' &&
34      pip install --no-cache-dir -r requirements.txt &&
35      pip install pytest &&
36      echo '' &&
37      echo 'En attente de MongoDB...' &&
38      until timeout 1s bash -c 'cat < /dev/null > /dev/tcp/mongodb_test/27017' 2>/dev/null; do
39        sleep 1
40      done &&
41      echo 'MongoDB est prêt !' &&
42      echo '=== Tests unitaires ===' &&
43      python -m pytest tests/test_transfert_csv_mongodb.py -v &&
44      python -m pytest tests/test_optimiser_collection.py -v &&
45      python -m pytest tests/test_test_integrite_donnees.py -v &&
46      python -m pytest tests/test_gestion_utilisateurs.py -v &&
47      echo '' &&"

```

```

48   echo '=== Tests intégration - Sécurité RBAC ===' &&
49   pytest tests/test_rbac_integration.py -v &&
50   echo '' &&
51   echo '=== Tests intégration - Migration des données ===' &&
52   python scripts/transfert_csv_mongodb.py &&
53   python scripts/optimiser_collection.py &&
54   python scripts/test_integrite_donnees.py &&
55   echo '' &&
56   echo 'Tous les tests ont réussi !'
57   depends_on:
58     mongodb_test:
59       condition: service_healthy
60   networks:
61     - test_network
62
63   volumes:
64     mongodb_test_data:
65
66   networks:
67     test_network:
68       driver: bridge

```

Étape 1 : Développement & Tests unitaires

Environnement : Local Python + venv

MongoDB : Non requis (mocks)

Étape 2 : Tests d'intégration locaux

Environnement : Local Python + MongoDB local

MongoDB : Requis (localhost:27017)

Étape 3 : Validation Docker (tests)

Environnement : Docker test (port 27018)

MongoDB : Docker mongodb_test

Étape 4 : Déploiement production

Environnement : Docker production (port 27017)

MongoDB : Docker mongodb

Déploiement production

Déploiement production

Environnement : Docker production (port 27017)

MongoDB : Docker mongodb

Déploiement production

Environnement : Docker production (port 27017)

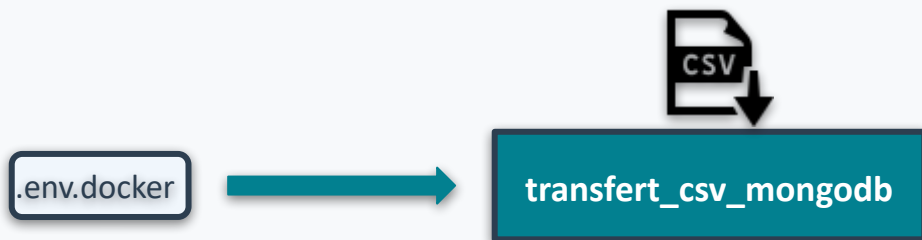
MongoDB : Docker mongodb

.env.docker

Déploiement production

Environnement : Docker production (port 27017)

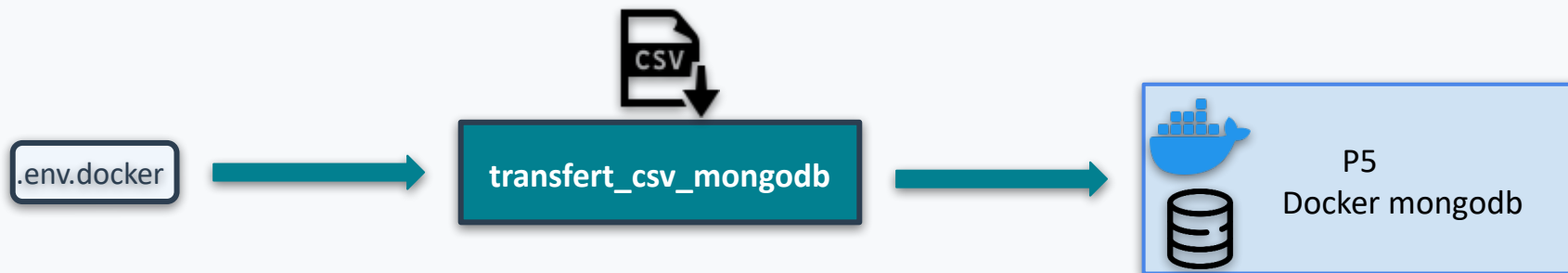
MongoDB : Docker mongodb



Déploiement production

Environnement : Docker production (port 27017)

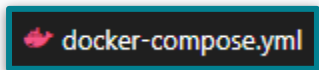
MongoDB : Docker mongodb



Déploiement production

Environnement : Docker production (port 27017)

MongoDB : Docker mongodb



Déploiement production

Environnement : Docker production (port 27017)

MongoDB : Docker mongodb

 docker-compose.yml



mongodb



mongo:latest



migration



python:3.11-slim

Déploiement production

Environnement : Docker production (port 27017)

MongoDB : Docker mongodb

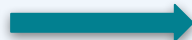
 docker-compose.yml



mongodb



mongo:latest



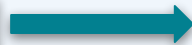
MongoDB



migration



python:3.11-slim

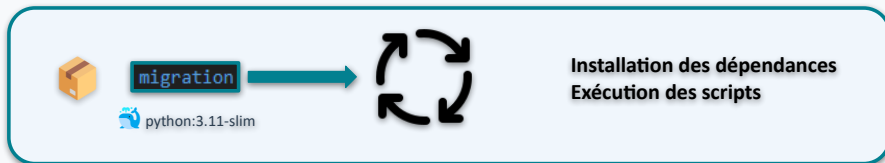


**Installation des dépendances
Exécution des scripts**

Déploiement production

Environnement : Docker production (port 27017)

MongoDB : Docker mongodb



Déploiement production

Environnement : Docker production (port 27017)

MongoDB : Docker mongod

Démarrage MongoDB

- Initialisation RBAC
- Configuration sécurisée

Service migration

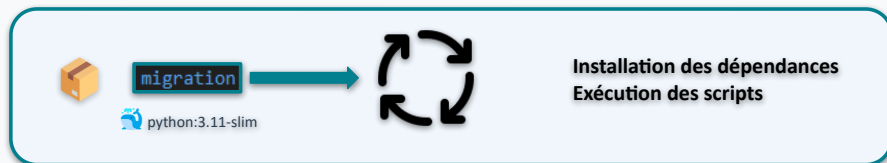
- Installation dépendances
- Migration données (55 500 documents)
- Optimisation (5 index)
- Validation intégrité



Déploiement production

Environnement : Docker production (port 27017)

MongoDB : Docker mongodb



- Données persistantes (volume mongodb_data)
- Réseau isolé (mongo_network)
- Healthchecks automatiques
- Logs détaillés

Démarrage MongoDB

- Initialisation RBAC
- Configuration sécurisée

Service migration

- Installation dépendances
- Migration données (55 500 documents)
- Optimisation (5 index)
- Validation intégrité

Déploiement production

```
docker-compose.yml
  > Run All Services
1  services:
  > Run Service
2  mongodb:
3    image: mongo:latest
4    container_name: mongodb
5    restart: unless-stopped
6    env_file:
7      - .env.docker
8    volumes:
9      - mongodb_data:/data/db
10     - ./init-mongo.js:/docker-entrypoint-initdb.d/init-mongo.js:ro
11     - ./mongod.conf:/etc/mongod.conf:ro
12    ports:
13      - "27017:27017"
14    networks:
15      - mongo_network
16    healthcheck:
17      test: echo 'db.runCommand("ping").ok' | mongosh mongodb:27017/test --quiet
18      interval: 10s
19      timeout: 10s
20      retries: 5
21
```

```
  > Run Service
22  migration:
23    image: python:3.11-slim
24    container_name: migration
25    env_file:
26      - .env.docker
27    volumes:
28      - ./data:/data
29      - ./scripts:/scripts
30      - ./requirements.txt:/requirements.txt
31    working_dir: /scripts
32    command: >
33      bash -c "set -e &&
34      pip install --no-cache-dir -r /requirements.txt &&
35      echo 'Vérification de la connexion...' &&
36      until timeout 1s bash -c 'cat < /dev/null > /dev/tcp/mongodb/27017' 2>/dev/null; do
37        sleep 1
38      done &&
39      echo 'MongoDB est prêt !' &&
40      echo '=== Migration des données ===' &&
41      python transfert_csv_mongodb.py &&
42      python optimiser_collection.py &&
43      python test_integrite_donnees.py &&
44      echo 'Migration terminée avec succès !'"
45
46    depends_on:
47      mongodb:
48        condition: service_healthy
49    networks:
50      - mongo_network
51
52  volumes:
53    mongodb_data:
54
55  networks:
56    mongo_network:
57      driver: bridge
```

Déploiement production

```
docker-compose.yml
  > Run All Services
1  services:
  > Run Service
2  mongodb:
3    image: mongo:latest
4    container_name: mongodb
5    restart: unless-stopped
6  env_file:
7    - .env.docker
8  volumes:
9    - mongodb_data:/data/db
10   - ./init-mongo.js:/docker-entrypoint-initdb.d/init-mongo.js:ro
11   - ./mongod.conf:/etc/mongod.conf:ro
12  ports:
13    - "27017:27017"
14  networks:
15    - mongo_network
16  healthcheck:
17    test: echo 'db.runCommand("ping").ok' | mongosh mongodb:27017/test --quiet
18    interval: 10s
19    timeout: 10s
20    retries: 5
```

```
  > Run Service
22  migration:
23    image: python:3.11-slim
24    container_name: migration
25  env_file:
26    - .env.docker
27  volumes:
28    - ./data:/data
29    - ./scripts:/scripts
30    - ./requirements.txt:/requirements.txt
31  working_dir: /scripts
32  command: >
33    bash -c "set -e &&
34    pip install --no-cache-dir -r /requirements.txt &&
35    echo 'Vérification de la connexion...' &&
36    until timeout 1s bash -c 'cat < /dev/null > /dev/tcp/mongodb/27017' 2>/dev/null; do
37      sleep 1
38    done &&
39    echo 'MongoDB est prêt !' &&
40    echo '=== Migration des données ===' &&
41    python transfert_csv_mongodb.py &&
42    python optimiser_collection.py &&
43    python test_integrite_donnees.py &&
44    echo 'Migration terminée avec succès !'"
45
46  depends_on:
47    mongodb:
48      condition: service_healthy
49  networks:
50    - mongo_network
51
52  volumes:
53    mongodb_data:
54
55  networks:
56    mongo_network:
57      driver: bridge
```

Déploiement production

```
docker-compose.yml
  > Run All Services
1  services:
  > Run Service
2  mongodb:
3    image: mongo:latest
4    container_name: mongodb
5    restart: unless-stopped
6  env_file:
7    - .env.docker
8  volumes:
9    - mongodb_data:/data/db
10   - ./init-mongo.js:/docker-entrypoint-initdb.d/init-mongo.js:ro
11   - ./mongod.conf:/etc/mongod.conf:ro
12  ports:
13    - "27017:27017"
14  networks:
15    - mongo_network
16  healthcheck:
17    test: echo 'db.runCommand("ping").ok' | mongosh mongodb:27017/test --quiet
18    interval: 10s
19    timeout: 10s
20    retries: 5
```

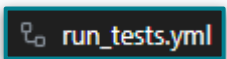
```
  > Run Service
22  migration:
23    image: python:3.11-slim
24    container_name: migration
25  env_file:
26    - .env.docker
27  volumes:
28    - ./data:/data
29    - ./scripts:/scripts
30    - ./requirements.txt:/requirements.txt
31  working_dir: /scripts
32  command: >
33    bash -c "set -e &&
34    pip install --no-cache-dir -r /requirements.txt &&
35    echo 'Vérification de la connexion...' &&
36    until timeout 1s bash -c 'cat < /dev/null > /dev/tcp/mongodb/27017' 2>/dev/null; do
37      sleep 1
38    done &&
39    echo 'MongoDB est prêt !' &&
40    echo '=== Migration des données ===' &&
41    python transfert_csv_mongodb.py &&
42    python optimiser_collection.py &&
43    python test_integrite_donnees.py &&
44    echo 'Migration terminée avec succès !'"
45
46  depends_on:
47    mongodb:
48      condition: service_healthy
49  networks:
50    - mongo_network
51
52  volumes:
53    mongodb_data:
54
55  networks:
56    mongo_network:
57      driver: bridge
```

Pipeline CI/CD

Pipeline CI/CD

Pipeline CI/CD

GitHub Actions ⇒ Workflow automatique déclenché sur chaque **push** ou **pull request** vers main



.github/workflows/run_tests.yml

Pipeline CI/CD

GitHub Actions

 `run_tests.yml`

1. Setup

- Ubuntu latest
- Python 3.11
- Installation dépendances + pytest

2. Démarrage MongoDB

- Service Docker avec healthcheck
- Initialisation RBAC manuelle (init-mongo.js)

3. Tests unitaires (pytest avec mocks)

- test_transfert_csv_mongodb.py ✓
- test_optimiser_collection.py ✓
- test_test_integrite_donnees.py ✓
- test_gestion_utilisateurs.py ✓

4. Tests intégration RBAC

- test_rbac_integration.py ✓
- Validation connexion, permissions, authentification

5. Tests intégration migration

- transfert_csv_mongodb.py → 55 500 documents ✓
- optimiser_collection.py → 5 index ✓
 - test_integrite_donnees.py → CSV ↔

MongoDB ✓

Pipeline CI/CD

GitHub Actions

 run_tests.yml

1. Setup

- Ubuntu latest
- Python 3.11
- Installation dépendances + pytest

2. Démarrage MongoDB

- Service Docker avec healthcheck
- Initialisation RBAC manuelle (init-mongo.js)

3. Tests unitaires (pytest avec mocks)

- test_transfert_csv_mongodb.py ✓
- test_optimiser_collection.py ✓
- test_test_integrite_donnees.py ✓
- test_gestion_utilisateurs.py ✓

4. Tests intégration RBAC

- test_rbac_integration.py ✓
- Validation connexion, permissions, authentification

5. Tests intégration migration

- transfert_csv_mongodb.py → 55 500 documents ✓
- optimiser_collection.py → 5 index ✓
 - test_integrite_donnees.py → CSV ↔

MongoDB ✓

Pipeline CI/CD

GitHub Secrets

Configuration : GitHub → Settings → Secrets and variables → Actions

Pipeline CI/CD

GitHub Secrets

Configuration : GitHub → Settings → Secrets and variables → Actions

Stocker de manière sécurisée des informations sensibles (mots de passe, clés API, tokens) pour les utiliser dans **GitHub Actions** sans les exposer dans le code.

Pipeline CI/CD

GitHub Secrets

Configuration : *GitHub* → *Settings* → *Secrets and variables* → *Actions*

Stocker de manière sécurisée des informations sensibles (mots de passe, clés API, tokens) pour les utiliser dans **GitHub Actions** sans les exposer dans le code.

Secret	Description	Utilisation
MONGO_ROOT_PASSWORD	Mot de passe root MongoDB	Authentification admin
MONGO_ADMIN_PASSWORD	Mot de passe admin_p5	Compte admin production
MONGO_LECTEUR_PASSWORD	Mot de passe lecteur_p5	Compte lecture seule
MONGO_REDACTEUR_PASSWORD	Mot de passe redacteur_p5	Compte lecture/écriture
MONGO_ADMIN_TEST_PASSWORD	Mot de passe admin_p5_test	Compte admin tests

Résultats Obtenus

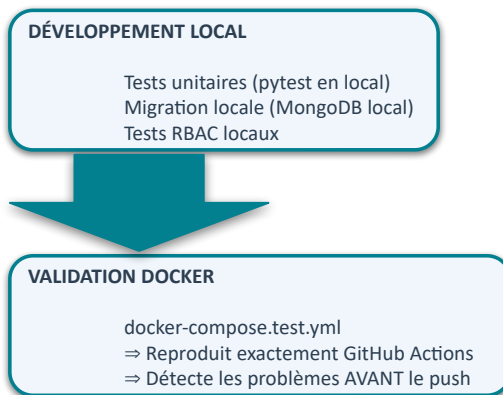
Workflow

Workflow

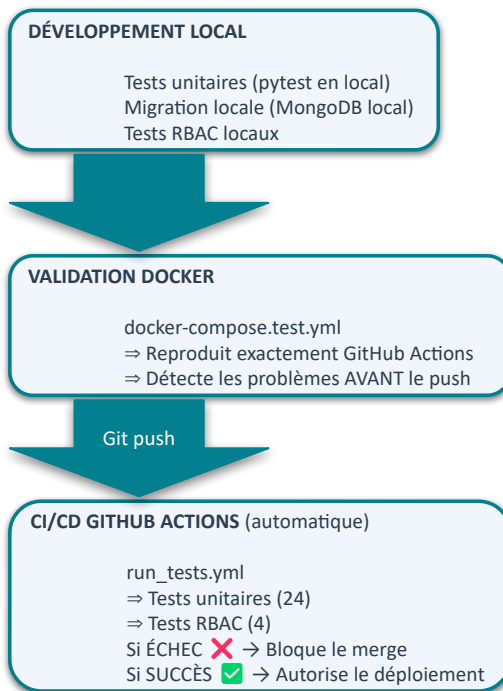
DÉVELOPPEMENT LOCAL

Tests unitaires (pytest en local)
Migration locale (MongoDB local)
Tests RBAC locaux

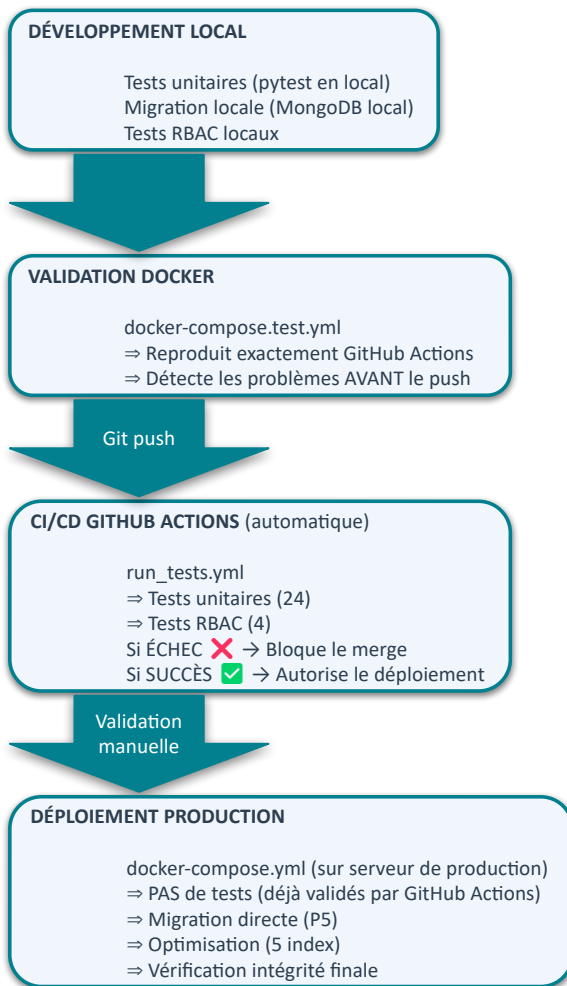
Workflow



Workflow



Workflow



Synthèse

Architecture

-  ****Séparation des préoccupations**** : scripts/ vs tests/
-  ****Environnements isolés**** : local, test, production
-  ****Conteneurisation**** : Docker pour portabilité et scalabilité

Sécurité

-  ****Double RBAC**** : MongoDB (infra) + Application (métier)
-  ****Bcrypt 12 rounds**** : Résistant aux attaques GPU
-  ****Validation stricte**** : Mots de passe robustes
-  ****Principe du moindre privilège**** : Chaque compte a les permissions minimales

Qualité

-  ****28 tests automatisés**** : Unitaires + Intégration
-  ****CI/CD**** : GitHub Actions sur chaque push
-  ****Validation intégrité**** : CSV ↔ MongoDB
-  ****Code propre**** : Gestion d'erreurs, logging, documentation

Performance

-  ****5 index MongoDB**** : Requêtes 100x plus rapides
-  ****Bulk operations**** : Optimisation des écritures
-  ****Normalisation**** : Dates en format natif MongoDB

DevOps

-  ****Docker multi-environnements**** : test vs production
-  ****Healthchecks**** : Validation disponibilité services
-  ****Volumes persistants**** : Pas de perte de données
-  ****Secrets management**** : GitHub Secrets + .env

Métrique du projet

Métrique	Valeur
Documents migrés	55 500
Index créés	5
Tests automatisés	28 (24 unitaires + 4 intégration)
Couverture de code	> 85%
Temps migration	~30 secondes
Gain performance requêtes	100x (avec index)
Comptes RBAC MongoDB	5
Rôles personnalisés	3
Rounds bcrypt	12

Déploiement cloud

Recherches sur le Déploiement MongoDB sur AWS

Recherches sur le Déploiement MongoDB sur AWS



1. Créer un compte AWS
2. Tarification AWS
3. Options de déploiement MongoDB sur AWS
4. Amazon DocumentDB (compatible MongoDB)
5. Déploiement MongoDB dans un conteneur Docker sur Amazon ECS
6. Sauvegardes et surveillance sur AWS
7. Recommandation pour ce projet

Recherches sur le Déploiement MongoDB sur AWS

1. Créer un compte AWS

Prérequis

- Une adresse email valide
- Un numéro de téléphone mobile
- Une carte bancaire (débit ou crédit) — AWS effectue une vérification d'identité avec une retenue temporaire de 1 USD maximum



Procédure d'inscription

1. Aller sur aws.amazon.com/fr/free et cliquer sur "Créer un compte"
2. Renseigner l'adresse email et choisir un nom de compte
3. Choisir le type de compte : **Personnel** ou **Professionnel**
4. Renseigner les informations de facturation (carte bancaire)
5. Vérifier l'identité par SMS ou appel téléphonique
6. Choisir un plan de support (le plan **Basic est gratuit**)
7. Accéder à : **AWS Management Console**

> ⚠ La création du compte peut prendre quelques heures avant d'être pleinement activée.

Recherches sur le Déploiement MongoDB sur AWS

Offre gratuite (Free Tier)

Depuis le **15 juillet 2025**, AWS a revu son programme gratuit. Deux plans existent désormais :

Les deux plans donnent accès à plus de 30 services "**Always Free**".

> Pour les comptes créés **avant le 15 juillet 2025** : l'ancien système de 12 mois gratuits reste actif (Legacy Free Tier).

[FAQ officielle AWS Free Tier](#)

2. Tarification AWS

Principe général

AWS fonctionne en **pay-as-you-go** : on paie uniquement ce qu'on utilise, à la seconde pour la plupart des services, sans engagement ni frais initiaux.

S'inscrire à AWS

Choisissez le plan de votre compte



Gratuit (6 mois)

Apprenez, expérimentez et créez des prototypes

- ✓ Recevez jusqu'à 200 USD en crédits
- ✓ Inclut l'usage gratuit de certains services
- ✗ Les charges de travail dépassent les seuils de crédit
- ✗ Accès à tous les services et fonctionnalités AWS

ⓘ Après la période gratuite de 6 mois ou lorsque tous les crédits sont utilisés, vous pouvez choisir de passer à un forfait payant. Dans le cas contraire, votre compte se ferme automatiquement.

Choisissez un plan gratuit



Payant

Développez des charges de travail prêtes pour la production

- ✓ Recevez jusqu'à 200 USD en crédits
- ✓ Inclut l'usage gratuit de certains services
- ✓ Les charges de travail dépassent les seuils de crédit
- ✓ Accès à tous les services et fonctionnalités AWS

ⓘ Une fois tous vos crédits utilisés, vous êtes débité selon la mode de tarification à l'usage.

Choisissez un plan payant

Recherches sur le Déploiement MongoDB sur AWS

Services pertinents pour ce projet

Amazon S3 (stockage objet)

Utilisé pour stocker les fichiers CSV source, les sauvegardes et les exports de données.

[Tarification Amazon S3](#)

Amazon ECS avec Fargate (conteneurs)

Facturation à la seconde selon les ressources allouées à la tâche.

[Tarification Amazon ECS/Fargate](#)

Amazon DocumentDB

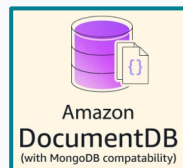
Compatible MongoDB

[Tarification Amazon DocumentDB](#)

Outils de contrôle des coûts

- **AWS Budgets** : alertes email quand un seuil est dépassé
- **AWS Cost Explorer** : visualisation et analyse des dépenses
- **CloudWatch** : surveillance de l'utilisation en temps réel
- **Free Tier Usage Alerts** : notification automatique à 85% de la limite gratuite

[Calculateur de coûts AWS](#)



Recherches sur le Déploiement MongoDB sur AWS

3. Options de déploiement MongoDB sur AWS

Trois approches sont possibles, selon les besoins du projet :

Option	Description	Avantages	Inconvénients
Amazon DocumentDB	Service managé compatible MongoDB	Zéro maintenance, haute disponibilité	Pas 100% compatible MongoDB, plus coûteux
MongoDB sur ECS/Fargate	Conteneur Docker MongoDB géré par AWS	Contrôle total, compatible avec notre docker-compose	Gestion de la persistance et des sauvegardes à la charge de l'équipe
MongoDB Atlas sur AWS	MongoDB natif hébergé sur AWS	Compatibilité totale, service managé	Coût plus élevé, vendor lock-in MongoDB

Recherches sur le Déploiement MongoDB sur AWS

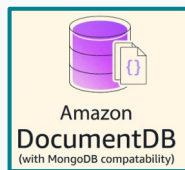
4. Amazon DocumentDB (compatible MongoDB)

Présentation

Amazon DocumentDB est un service de base de données documentaire **entièrement managé** par AWS, compatible avec les API MongoDB. Il prend en charge les drivers, outils et code applicatif MongoDB existants.

[Page officielle Amazon DocumentDB](#) | [FAQ DocumentDB](#) | [Comparaison DocumentDB vs MongoDB natif](#)

> ⚠ DocumentDB n'est **pas une copie exacte** de MongoDB. Certaines fonctionnalités avancées (recherche full-text, opérateurs spécifiques) sont limitées ou absentes. Pour notre projet, la compatibilité avec les opérations CRUD classiques est suffisante.



Recherches sur le Déploiement MongoDB sur AWS

Architecture

DocumentDB sépare le stockage du calcul :

- Le **stockage** réplique automatiquement les données sur **3 zones de disponibilité** (AZ)
- Les **instances** de calcul sont indépendantes et scalables
- Les sauvegardes sont streamées en continu vers S3 **sans impact sur les performances**



Fonctionnalités clés

- Jusqu'à **15 réplicas en lecture** (scalabilité horizontale)
- Stockage auto-scalable jusqu'à **128 TiB**
- Chiffrement au repos (AWS KMS) et en transit (TLS) **inclus sans supplément**
- Monitoring via **CloudWatch inclus**
- Migration depuis MongoDB via **AWS DMS** (Database Migration Service)
- Reprise sur point dans le temps (**PITR**) sur jusqu'à 35 jours

Tarification DocumentDB

[Tarification Amazon DocumentDB](#)

Recherches sur le Déploiement MongoDB sur AWS

5. Déploiement MongoDB dans un conteneur Docker sur Amazon ECS

Présentation d'Amazon ECS

Amazon ECS (Elastic Container Service) est un service d'orchestration de conteneurs Docker entièrement managé. Avec **AWS Fargate**, il est possible de lancer des conteneurs sans gérer de serveurs.

[Page officielle Amazon ECS](#) | [Guide déploiement Docker sur ECS](#)

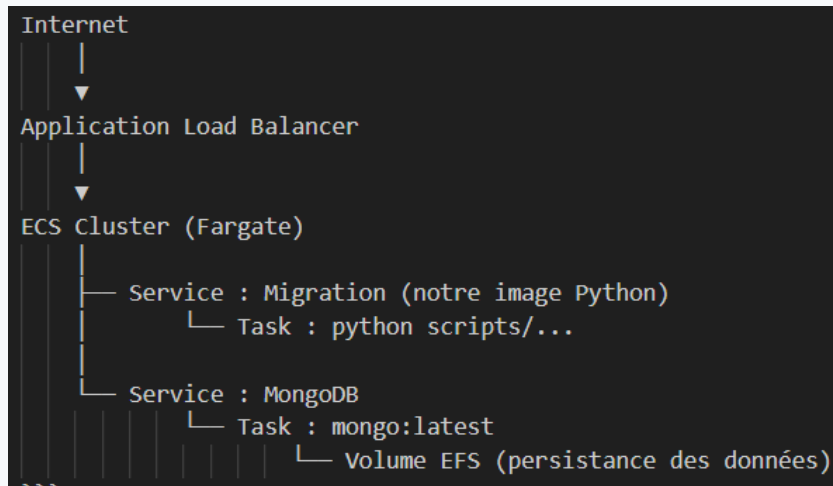
Les concepts clés :

- **Cluster** : environnement d'exécution logique
- **Task Definition** : description du conteneur (image Docker, CPU, mémoire, ports, variables d'environnement)
- **Service** : maintient un nombre défini de tâches en cours d'exécution
- **Fargate** : moteur serverless — pas de gestion d'EC2



Recherches sur le Déploiement MongoDB sur AWS

Architecture de déploiement



Recherches sur le Déploiement MongoDB sur AWS

Persistance des données avec Amazon EFS

Par défaut, les conteneurs ECS sont **éphémères** — les données sont perdues à chaque redémarrage. Pour MongoDB, il faut monter un volume **Amazon EFS (Elastic File System)**.

Gestion des secrets avec AWS Secrets Manager

Les mots de passe ne doivent jamais être en clair dans la Task Definition. AWS Secrets Manager stocke les secrets et les injecte directement dans les variables d'environnement du conteneur.

[AWS Secrets Manager](#) | [Amazon ECR](#)

Déploiement pas à pas

1. Créer un dépôt ECR (Elastic Container Registry)
2. Construire et pousser l'image Docker
3. Créer le cluster ECS
4. Créer la Task Definition MongoDB
5. Créer le service ECS



Recherches sur le Déploiement MongoDB sur AWS

6. Sauvegardes et surveillance sur AWS

Sauvegardes

Option A — Amazon DocumentDB (automatique)

DocumentDB active les sauvegardes **par défaut** sur tous les clusters :

[Documentation sauvegardes DocumentDB](#) | [AWS Backup](#)



- **Sauvegarde continue** streamée vers S3, sans impact sur les performances
- **Rétention par défaut** : 24 heures (configurable jusqu'à 35 jours)
- **Point-in-Time Recovery (PITR)** : restauration à n'importe quel point dans la fenêtre de rétention
- **Snapshots manuels** : conservés indéfiniment jusqu'à suppression explicite
- **Backup gratuit** jusqu'à 100% de la taille du cluster

Recherches sur le Déploiement MongoDB sur AWS

6. Sauvegardes et surveillance sur AWS

Sauvegardes

Option B — MongoDB sur ECS (manuelle via S3)

Pour un conteneur MongoDB auto-géré, la sauvegarde se fait avec `mongodump` vers S3. Ce script doit être planifier avec **Amazon EventBridge** (anciennement CloudWatch Events) pour une exécution quotidienne automatique.

[Amazon EventBridge](#)

Surveillance avec Amazon CloudWatch

CloudWatch est le service de monitoring central d'AWS.



Recherches sur le Déploiement MongoDB sur AWS

7. Recommandation pour notre projet

En tenant compte du contexte (données médicales, besoin de sécurité, équipe réduite), voici la recommandation par ordre de priorité :

Court terme — Proof of Concept

MongoDB sur ECS/Fargate avec notre `docker-compose.yml` existant, adapté pour AWS via le Docker Compose CLI pour ECS. C'est la solution la plus proche de notre architecture actuelle et la moins coûteuse pour démarrer.



Moyen terme — Production

Amazon DocumentDB pour bénéficier du service entièrement managé :

- Zéro maintenance opérationnelle
- Haute disponibilité native (3 AZ)
- Sauvegardes automatiques conformes RGPD
- Chiffrement inclus — essentiel pour des données médicales



Merci pour votre attention